

Evaluating the Synthetic Engineer: A Comprehensive Framework for Measuring AI Utilization and its Multidimensional Effects on Software Engineering

The rapid integration of Large Language Models (LLMs) and agentic workflows into the software development life cycle has moved beyond a phase of experimental curiosity into a period of institutional adoption. As organizations scale their investments in tools such as GitHub Copilot, Cursor, and various autonomous agents, the necessity for a rigorous, data-driven framework to evaluate both the utilization and the resulting impact of these technologies has become paramount. Traditional engineering metrics, while foundational, often fail to account for the unique socio-technical shifts introduced by machine-generated code, such as the acceleration of the "inner loop" at the expense of "outer loop" stability, and the emergence of comprehension-based technical debt. This report establishes a dual-layered measurement paradigm, distinguishing between the indicators of AI usage and the subsequent effects on system health, delivery velocity, and organizational knowledge.

Pillar I: The Metrics of AI Utilization and Adoption

The first critical dimension of measurement focuses on the breadth and depth of AI tool integration within the daily routines of the engineering staff. These metrics serve as leading indicators of organizational maturity and are essential for identifying where adoption has plateaued or where the toolchain is failing to provide sufficient value. Utilization metrics are primarily derived from IDE telemetry, repository metadata, and administrative API logs provided by AI vendors.

Behavioral Adoption and Habituation

The transition to AI-assisted development is rarely uniform. Tracking the consistency of use reveals the degree to which a tool has been folded into the reflexive problem-solving habits of a developer.¹ If a tool is only used sporadically, it suggests that the overhead of the tool—such as prompt engineering or the correction of hallucinations—may outweigh the perceived benefits.

Worksheet 1: AI Utilization and Engagement Metrics	Definition and Measurement Source	Context and Significance
--	-----------------------------------	--------------------------

Daily Active Users (DAU)	The count of unique developers invoking AI features in a 24-hour window. Measured via IDE Plugin Telemetry. ²	Indicates how many engineers rely on AI as a prerequisite for their daily coding tasks. ¹
Weekly/Monthly Active Users (WAU/MAU)	The count of unique users over 7 or 28 days. Measured via Vendor Dashboard. ²	Captures broad adoption and helps distinguish between experimental trials and habit formation. ²
Seat Utilization Rate	The ratio of licensed developers to active users. Measured via License Management APIs. ³	Identifies licensing waste and signals friction in the initial onboarding or setup process. ³
Session Depth	The intensity of interaction per session, distinguishing single-line completions from multi-turn refactors. Measured via IDE Event Logs. ²	Deep sessions correlate with high-value tasks like architecture or debugging, while shallow use reflects simple boilerplate. ²
Retention Rate	The percentage of developers who continue using the tool 30, 60, and 90 days post-onboarding. Measured via Longitudinal Usage Tracking. ²	High drop-off rates often precede negative productivity outcomes due to low trust or poor context quality. ²
Invocation Counts	The raw frequency of AI feature calls (completions, chat, agents) per developer. Measured via CLI/IDE Telemetry. ²	Provides a volume-based signal of engagement across different modes of assistance. ²

The interaction between Daily Active Users and the total licensed count is often the most telling signal of a tool's internal reputation. An organization may report high MAU, indicating that everyone has tried the tool, while a low DAU suggests that the team does not trust the tool for core production work.² Furthermore, monitoring the "Tool Diversity Index"—the variety of distinct AI agents used for different stages of the SDLC—provides insight into whether the organization is stuck in the "coding assistant" phase or has moved into "agentic orchestration,"

where AI is used for PR reviews, security scanning, and documentation.²

Telemetry of Synthetic Contributions

Beyond mere activity, measuring the actual output of the AI during the development session provides a quantitative look at the "synthetic share" of the codebase. This involves tracking the volume of suggestions offered versus those accepted and committed.

Worksheet 2: Synthetic Code Generation and Acceptance	Metric Calculation and Logic	Operational Implication
Acceptance Rate	(Accepted AI suggestions ÷ Total AI suggestions) × 100. Benchmarked at ~27–30% for general code. ³	Measures the alignment between the AI's output and the developer's immediate needs and standards. ¹
AI Touch Ratio	(AI-generated lines ÷ Total lines changed in a PR) × 100. Measured via Git Metadata. ⁸	Shows the influence of AI on specific pull requests; high-performing teams target 40–60%. ⁸
Prompt→Commit Success Rate	(AI suggestions shipped without human rewrite ÷ Total suggestions) × 100. Measured via Repository Diffs. ⁴	Higher rates indicate well-crafted prompts and growing trust, while drops warn of model drift or hallucinations. ⁴
AI interaction Time	Total duration spent in the AI interface or chat. Measured via Browser/IDE logs. ²	Helps evaluate if the AI is a productivity booster or a source of "distraction toil". ²
Reflexive Turn Frequency	The rate at which a developer uses AI for new problems. Measured via Intent-based telemetry. ¹	Measures the behavioral shift from manual research to AI-first problem solving. ¹

Analyzing these metrics requires a nuanced understanding of the "AI Mirror Effect," where the tools tend to amplify existing organizational capabilities.⁸ For instance, a high acceptance rate in a team with weak testing and review cultures may not signal success, but rather a dangerous bypass of verification steps.¹⁰ Conversely, in highly regulated environments, lower acceptance

rates may be expected as developers exercise more rigorous oversight.¹²

Qualitative Indicators of Utilization

Human-centric metrics collected through surveys and sentiment analysis provide context that telemetry often misses. These indicators focus on the psychological state of the engineer and the perceived value of the AI integration.¹

Worksheet 3: Sentiment and Psychological Use Metrics	Definition and Assessment Method	Strategic Value
Trust Level	Self-assessed confidence in AI accuracy. Measured via Developer Surveys. ¹	Identifies if skepticism is hindering adoption or if "blind following" is creating risk. ¹
Reliance Level	Perceived dependency on AI to complete daily work. Measured via Qualitative Interviews. ¹	Monitors for potential skill atrophy or high-dependency risks in the engineering staff. ¹
Personal Ownership	The psychological feeling of owning the code and its quality. Measured via Sentiment Polling. ¹	Prevents the dilution of accountability as the volume of AI-generated code increases. ¹
Job Satisfaction	Fulfilment levels in the role post-AI adoption. Measured via eNPS or Wellness Surveys. ¹	Ensures AI automation is reducing toil rather than creating "mechanized burnout". ¹

The risk of "craft alienation" is a growing concern among senior engineers who may find that the tactile experience of shaping code is being replaced by a more detached "editor-in-chief" role.¹⁵ Monitoring satisfaction and ownership metrics allows leadership to balance speed gains with the long-term health and retention of their talent pool.

Pillar II: The Metrics of AI's Effect on Software Engineering

The second pillar of the framework evaluates the downstream consequences of AI adoption. While utilization tells us if the tools are being used, the "effect" metrics tell us if that usage is

actually beneficial to the business. This includes impacts on delivery velocity, code quality, security posture, and the financial bottom line.

Impact on Delivery Velocity and Throughput

AI is widely recognized as an accelerator for the "inner loop"—the activities of coding and unit testing.¹⁷ However, the systemic impact on the "outer loop"—integration, deployment, and release—is more complex. When coding speeds up, the constraints often shift to code reviews and integration pipelines.¹⁸

Worksheet 4: Velocity and Throughput Outcomes	Metric Definition and AI Influence	Expected Outcome with AI
Cycle Time	Median time from first commit to production release. Measured via Version Control Systems. ⁴	Targeted reduction of 15–25% as the initial coding phase is compressed. ⁴
PR Throughput	Count of merged pull requests per engineer per week. Measured via GitHub/GitLab API. ⁴	High AI users typically merge significantly more PRs (up to 5x) than non-users. ²⁰
TrueThroughput	Output count adjusted for PR complexity using AI analysis. Measured via DX Platform. ²¹	Provides a more accurate signal of engineering output than raw PR counts. ²¹
Deployment Frequency	How often the team releases to production. Measured via CI/CD Logs. ¹	Often increases by 20–30% as AI facilitates smaller, more frequent updates. ⁴
Lead Time for Changes	Time from commit to running in production. Measured via Deployment Logs. ¹	Elite teams target < 1 hour; AI-fluent teams often achieve these benchmarks. ²²

The dramatic increase in PR volume observed in AI-enabled teams has a ripple effect on the "Pull Request Cycle Time." If the human review capacity does not scale alongside the machine generation capacity, PRs may sit in "stalled" states for longer periods.¹⁰ This creates a "Productivity Paradox," where developers feel 20% faster but the actual delivery of features remains stagnant due to downstream bottlenecks.¹⁹

Quality, Stability, and Security Outcomes

The most contentious area of AI impact is code quality. Research suggests that while AI can help generate tests and documentation, it also introduces a higher volume of logic errors and security vulnerabilities compared to human-only code.¹²

Worksheet 5: Quality and Stability Metrics	Metric and Calculation	AI Impact Observations
Defect Density	(Confirmed bugs ÷ KLOC). Split by AI-assisted vs. human-written code. ⁴	AI-assisted code often shows ~1.7x higher defect density than manual implementations. ⁹
Change Failure Rate (CFR)	% of deployments causing production failures. Measured via Incident Management. ¹	CFR may rise initially as teams adapt to AI output; stable or declining CFR signals maturity. ¹⁰
PR Revert Rate	(Reverted PRs ÷ Total PRs). Measured via VCS. ²¹	A critical lagging indicator; spikes suggest that speed is being prioritized over quality. ²¹
AI Revert Percentage	(AI lines reverted ÷ AI lines merged). Measured via Git history. ⁴	Novel internal metric focusing specifically on AI-induced safety failures. ⁴
Rework Rate	% of code modified or fixed within 30 days of merge. Measured via Commit history. ¹	AI-generated code often shows a rework rate of 35–40%, nearly double the human baseline. ⁹
Security Vulnerability Density	Count of CVSS 7.0+ flaws per module. Measured via SAST/SCA tools. ¹²	AI code has been found to contain 2.74x more vulnerabilities than human code. ¹²
Secret Leakage Rate	% of commits containing hardcoded credentials. Measured via Gitleaks/Vault	AI-assisted commits show ~3.2% leak rates versus a 1.5% baseline in some

	Radar. ¹²	environments. ¹²
--	----------------------	-----------------------------

Language-specific patterns further complicate the quality landscape. For example, Java has been identified as having one of the highest security failure rates (72%) when generated by AI, while Python performs significantly better with a 62% security pass rate.¹² This variability necessitates language-specific quality gates and review protocols.

The Accumulation of Comprehension Debt

A novel and particularly insidious effect of AI is "Comprehension Debt"—the gap between the volume of code in a system and the amount that human maintainers genuinely understand.¹¹ This is a "high-interest" variant of technical debt that accumulates invisibly during periods of high velocity.

Worksheet 6: Comprehension and Cognitive Debt Metrics	Metric and Calculation	Significance for Engineering Leaders
Revision Depth	(Lines changed after AI generation ÷ Total AI lines). Measured via Review history. ¹⁰	Measures the "correction tax"; high depth means AI is creating more work for reviewers. ¹⁰
Review Turnaround Time	Time from PR opening to first review and final merge. ¹⁰	Spikes in review time suggest that AI-generated code is harder to audit and understand. ¹⁰
Comprehension Quiz Score	Results from targeted quizzes on recent changes. Measured via Human Triage. ²⁹	Quantifies the "black box" effect; developers using AI often score 17% lower on these tests. ³⁰
Code Maintainability Score	Developer perception of how easy code is to modify. Measured via DX Surveys. ²⁵	Often declines with increased AI use as developers feel further removed from the logic. ²⁵
Incident Resolution Time (MTTR)	Time to restore service after AI-linked failures. Measured via Pager Logs. ¹	Measures if comprehension debt is making production issues harder to diagnose. ¹⁰

Comprehension debt typically follows a three-stage progression: the "Honeymoon" (days

1–30), where velocity spikes; the "Drift" (days 30–180), where institutional knowledge erodes; and the "Cliff" (day 180+), where the codebase becomes unmaintainable archaeology.²⁹ Tracking the "AI Touch Ratio" alongside "Onboarding Ramp Time" can help identify if this debt is being paid forward to future hires.

The Tooling Ecosystem for Metric Collection

To effectively capture these two pillars of metrics, organizations must deploy a multi-layered infrastructure that integrates usage telemetry with delivery outcomes.

Engineering Intelligence Platforms

Engineering intelligence platforms act as the central hub for AI measurement, connecting to VCS, ticket tracking, and CI/CD tools to provide a holistic view.

- **Axify:** This platform specializes in mapping delivery signals to AI usage. It allows teams to visualize trendlines for "Before/After Cycle Time" and "AI Output Acceptance Rate," providing a dashboard for both DORA and AI-specific indicators.¹⁰
- **DX (Developer Experience):** DX provides a research-based framework specifically designed to track the impact of AI code assistants. It uses the "AI Impact Reports" to visualize how AI usage impacts speed, effectiveness, and quality, and it offers the "Developer Experience Index (DXI)" as a composite measure of performance.²⁰
- **LinearB:** Focused on DevOps and delivery velocity, LinearB integrates with Jira and Git to track DORA metrics. It has introduced AI-driven features to audit engineering costs and identify friction in the code review lifecycle.³²
- **Pensero:** An AI-native platform that goes beyond quantitative output to understand the "substance" of engineering work. It assesses output quality over time and provides "AI Cycle Analysis" to show the real impact on work patterns.³³
- **Waydev:** Provides complete visibility into pull request activities and calculates AI ROI by comparing historical engineering data before and after adoption.³⁵

Specialized Monitoring and Governance Tools

Security and quality scanners are essential for monitoring the quality "effect" metrics and enforcing guardrails on AI output.

- **SonarQube / SonarCloud:** These tools serve as an independent verification layer for both AI and human code. They are particularly effective at identifying "technical debt interest" and ensure that AI-generated code adheres to team-specific naming conventions and architectural standards.³⁶
- **Semgrep:** Unifies SAST, SCA, and secrets scanning. Its AI-native detection reasoning is designed to filter out the noise typical of traditional static analyzers, which is critical when code volume increases.³⁹
- **Datadog Code Security:** Continuously scans repositories for leaked secrets and malicious code patterns, interpreting "intent" in a way that traditional pattern matching

often misses.⁴⁰

- **Gitleaks & Vault Radar:** Essential for the specific task of monitoring "Secret Leakage Rates" in real-time as developers commit code.²⁶
- **Confident AI:** A Git-based platform for prompt experimentation and evaluation. It allows teams to run metrics against changed prompts, ensuring that regressions are surfaced in the same workflow as code review.⁴¹

Economic Impact and Return on AI Investment (ROAI)

Measuring the effect of AI also requires a financial lens to justify the expenditure on licenses and API tokens. The "Return on AI Investment (ROAI)" combines quantitative efficiency gains with qualitative experience improvements.

Financial Performance Indicators

The economic value of AI in software engineering is typically assessed across four dimensions: capacity, quality, cost, and talent.¹⁰

Worksheet 7: Financial and ROAI Framework	Formula or Indicator	Business Value Context
ROI (%)	$((\text{Total Benefit} - \text{Total Cost}) / \text{Total Cost}) \times 100.$ ⁴	A benchmark case showed a 39x ROI based on time saved vs. license cost. ²⁰
Cost per Delivery Unit	Total team cost ÷ Units delivered (PRs or features). ¹⁰	Determines if AI tools reduce the unit cost of work over time. ¹⁰
Capacity Unlocked	Total developer hours saved per week × Hourly rate. ¹⁰	Industry averages suggest ~3.6 hours saved per developer per week. ²⁰
Agent Hourly Rate	Human-equivalent hours completed ÷ AI spend. ²⁰	Measures the efficiency of autonomous agents compared to human contractors. ²⁰
Token Utilization Cost	Average spend per developer per active day. ⁴³	Enterprise averages sit around \$13 per developer per active day for advanced agents. ⁴³

The McKinsey research suggests that the direct impact of AI on software engineering productivity could be between 20% to 45% of current annual spending.⁴⁴ However, organizations often face hidden costs—between 10% to 20% beyond budget—for technical debt remediation and integration challenges.⁴⁵

Onboarding and Talent Leverage

One of the most measurable financial benefits of AI is the compression of onboarding time, which reduces the "attrition tax" and accelerates the time-to-value for new hires.²²

Worksheet 8: AI-Accelerated Onboarding Benchmarks	Metric	Performance with AI Assistance
Time to 10th PR	Days from start date to 10th merged PR. ⁴⁶	Reduced from 91 days (no AI) to 33–49 days (daily AI use). ⁴⁶
Time to First Commit	Days to first repository contribution. ⁴⁷	Standard benchmark is ≤ 3 days; AI-fluent teams often hit < 4 hours. ⁴⁷
Time to Independence	Days to complete a non-trivial solo feature. ⁴⁸	Reduced from 4–8 weeks to < 2 weeks with AI-driven codebase understanding. ⁴⁸

This acceleration allows companies to deploy talent more dynamically, shifting engineers across projects with much lower context-switching costs.⁵⁰ However, leadership must monitor the "90-day Retention Rate" to ensure that the increased speed of onboarding does not lead to early burnout or a lack of connection to the team's culture.⁴⁷

Mathematical Modeling for Reliability and Risk

To deeply understand the "effect" of AI, engineering organizations are increasingly using mathematical models to predict system-level failures and debt accumulation.

The Reliability Chain Model

When software relies on a chain of services where each contains AI-generated logic, the failure probability is calculated using the series-reliability product ⁵¹:

$$P(failure) = 1 - (1 - \epsilon)^d$$

Where ϵ is the per-service error rate and d is the chain depth. For AI-generated code in 2026, a defensible empirical envelope for ϵ is 0.05 to 0.20.⁵¹ As d grows, the probability of failure increases rapidly; at $\epsilon = 0.05$ and $d = 10$, the failure probability is 40.1%.⁵¹

The AI-Adjusted Technical Debt Ratio (TDR)

The Technical Debt Ratio measures the cost of remediation against the cost of development. In the AI era, this formula must be adjusted for the "principal" of comprehension debt and the "interest" of maintenance³¹:

$$TDR(AI) = \frac{Remediation\ Cost + AI-Introduced\ Debt}{Development\ Cost - AI-Acceleration\ Benefit} \times 100$$

This model helps organizations visualize how AI may temporarily lower short-term TDR while simultaneously increasing the long-term "interest rate" of maintenance as understanding lags behind code growth.¹¹

Strategic Summary of Metrics and Tools

The effective measurement of AI in software engineering requires a balanced scorecard that looks at both leading (use) and lagging (effect) indicators. The following summary table provides a high-level mapping of metrics to their collection mechanisms.

Metric Category	Key Metrics	Primary Collection Tools
Utilization & Adoption	DAU/WAU, Acceptance Rate, AI Touch Ratio, Session Depth.	GitHub Copilot Dashboard, Axify, IDE Telemetry. ³
Delivery & Velocity	Cycle Time, Lead Time, PR Throughput, TrueThroughput.	LinearB, DX, Waydev, VCS APIs. ²¹
Quality & Security	Defect Density, Rework Rate, Secret Leakage, Vulnerability Count.	SonarQube, Semgrep, Datadog, Gitleaks. ³⁶
Socio-Technical Debt	Comprehension Quiz Scores, Revision Depth,	DX, Pensero, Human Triage, PagerDuty. ²¹

	MTTR, DX Index.	
Financial & ROI	Cost per Delivery Unit, Capacity Unlocked, Token Spend.	Finout, CloudZero, Claude Console, Prompts.ai. ⁵²

Conclusion and Recommendations

The transition to AI-assisted engineering is not a simple linear improvement in speed; it is a fundamental reconfiguration of the software development lifecycle. Organizations that measure only velocity will likely succumb to the "cliff" of comprehension debt and quality degradation. To achieve sustainable performance gains, leadership must prioritize a "vibe, then verify" culture, backed by rigorous measurement of both the use and the effect of AI.

The analysis indicates that the most successful organizations will be those that treat AI as a "thinking partner" rather than a "code factory," using metrics to protect the cognitive health and architectural understanding of their engineers while aggressively automating the toil of implementation.³⁷ Future measurement strategies should focus on the "Double Transformation"—technical and organizational—to ensure that the 20x leverage promised by agentic AI results in long-term business value rather than an unmanageable legacy of machine-generated complexity.¹⁶

Citerade verk

1. DORA metrics
2. Measuring AI Adoption & Tool Usage — What to Track Before You Code - Agile Analytics, hämtad maj 12, 2026, <https://www.agileanalytics.cloud/blog/measuring-ai-adoption-and-tool-usage-what-to-track-before-you-code>
3. How to Monitor GitHub Copilot Usage Across Your Engineering Team, hämtad maj 12, 2026, <https://jellyfish.co/library/github-copilot-monitoring/>
4. Autonomous Development Metrics: KPIs That Matter for AI-Assisted ..., hämtad maj 12, 2026, <https://www.augmentcode.com/tools/autonomous-development-metrics-kpis-that-matter-for-ai-assisted-engineering-teams>
5. How to Measure the ROI of AI Code Assistants in Software Development - Jellyfish, hämtad maj 12, 2026, <https://jellyfish.co/library/ai-in-software-development/measuring-roi-of-code-assistants/>
6. Data available in Copilot usage metrics - GitHub Enterprise Cloud Docs, hämtad maj 12, 2026, <https://docs.github.com/en/enterprise-cloud@latest/copilot/reference/copilot-usage-metrics/copilot-usage-metrics>
7. Experience with GitHub Copilot for Developer Productivity at Zoominfo - arXiv,

- hämtad maj 12, 2026, <https://arxiv.org/html/2501.13282v1>
8. GitHub Copilot Metrics: From Usage Data to Real Delivery Impact - Axify, hämtad maj 12, 2026, <https://axify.io/blog/github-copilot-metrics>
 9. How to Design Code Quality Metrics for AI Generated Code - Exceeds AI Blog, hämtad maj 12, 2026, <https://blog.exceeds.ai/ai-generated-code-quality-metrics/>
 10. 20 AI Performance Metrics You Should Follow in Software Development - Axify, hämtad maj 12, 2026, <https://axify.io/blog/ai-performance-metrics>
 11. Comprehension Debt in GenAI-Assisted Software Engineering Projects - arXiv, hämtad maj 12, 2026, <https://arxiv.org/html/2604.13277v1>
 12. Enterprise AI Coding Security Risks 2025: Complete Guide - Exceeds AI Blog, hämtad maj 12, 2026, <https://blog.exceeds.ai/ai-coding-assistants-risks-2025/>
 13. Best AI Coding Assistants 2026: Cursor vs Copilot vs Claude Code - Scrimba, hämtad maj 12, 2026, <https://scrimba.com/articles/best-ai-coding-assistants-2026/>
 14. What is Developer Velocity and Why It Matters? - Multitudes, hämtad maj 12, 2026, <https://www.multitudes.com/blog/developer-velocity>
 15. Comprehension Debt: The Hidden Tax on AI-Generated Code | jvaneyck - WordPress.com, hämtad maj 12, 2026, <https://jvaneyck.wordpress.com/2026/03/21/comprehension-debt-the-hidden-tax-on-ai-generated-code/>
 16. The AI revolution in software development - McKinsey, hämtad maj 12, 2026, <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-ai-revolution-in-software-development>
 17. Yes, you can measure software developer productivity - McKinsey, hämtad maj 12, 2026, <https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/yes-you-can-measure-software-developer-productivity>
 18. AI Coding Tools Metrics - TechEmpower, hämtad maj 12, 2026, <https://www.techempower.com/blog/2025/12/01/ai-coding-tools-metrics/>
 19. 2026 Engineering Productivity Benchmarks: What AI Is Really Changing in Software Delivery - Plandek, hämtad maj 12, 2026, <https://plandek.com/blog/what-ai-is-really-changing-in-software-delivery>
 20. How to measure AI's impact on developer productivity - DX, hämtad maj 12, 2026, <https://getdx.com/blog/ai-measurement-hub/>
 21. 5 metrics in DX for measuring the impact of AI on developer ..., hämtad maj 12, 2026, <https://getdx.com/blog/5-metrics-in-dx-to-measure-ai-impact/>
 22. McKinsey & Company: Measuring AI Developer Productivity & Attrition Risk 2026, hämtad maj 12, 2026, <https://codeninity.com/reports/mckinsey-ai-developer-productivity-2026>
 23. The Hidden Costs of AI-Generated Software: Why "It Works" Isn't Enough - Codebridge, hämtad maj 12, 2026, <https://www.codebridge.tech/articles/the-hidden-costs-of-ai-generated-software-why-it-works-isnt-enough>
 24. AI-Generated Code Security Risks - Why Vulnerabilities Increase 2.74x and How to Prevent Them - SoftwareSeni, hämtad maj 12, 2026,

- <https://www.softwareseni.com/ai-generated-code-security-risks-why-vulnerabilities-increase-2-74x-and-how-to-prevent-them/>
25. Three metrics for measuring the impact of AI on code quality - DX, hämtad maj 12, 2026,
<https://getdx.com/blog/3-metrics-for-measuring-the-impact-of-ai-on-code-quality/>
 26. Secrets Detection With AI: A Guide for Developers - IBM, hämtad maj 12, 2026,
<https://www.ibm.com/think/topics/secrets-detection-with-ai>
 27. AI Coding Security Vulnerability Statistics 2026: Alarming Data - SQ Magazine, hämtad maj 12, 2026,
<https://sqmagazine.co.uk/ai-coding-security-vulnerability-statistics/>
 28. True Cost of AI-Generated Code. A Strategic Analysis of "Comprehension..." - Medium, hämtad maj 12, 2026,
<https://medium.com/@justhamade/true-cost-of-ai-generated-code-f4362391790c>
 29. Comprehension Debt: The Hidden Cost of AI-Generated Code - Allstacks, hämtad maj 12, 2026,
<https://www.allstacks.com/blog/comprehension-debt-the-hidden-cost-of-ai-generated-code>
 30. Comprehension Debt - the hidden cost of AI generated code. - AddyOsmani.com, hämtad maj 12, 2026,
<https://addyosmani.com/blog/comprehension-debt/>
 31. Technical debt ratio: How to measure technical debt - DX, hämtad maj 12, 2026,
<https://getdx.com/blog/technical-debt-ratio/>
 32. Jira Metrics & Dashboards for Engineering Teams - LinearB, hämtad maj 12, 2026,
<https://linearb.io/integrations/jira>
 33. 10 Best Engineering Productivity Measurement Tools in 2026 - Pensero, hämtad maj 12, 2026, <https://pensero.ai/blog/engineering-productivity-measure-tools>
 34. Best Linear Integrations for Engineering Teams in 2026 - Pensero, hämtad maj 12, 2026, <https://pensero.ai/blog/linear-integrations>
 35. How to Measure AI ROI on Your Engineering Team - Waydev, hämtad maj 12, 2026, <https://waydev.co/how-to-measure-ai-roi-on-your-engineering-team/>
 36. Code Quality Tool Analysis & Standard Scanning Metrics Solution - Sonar, hämtad maj 12, 2026, <https://www.sonarsource.com/solutions/quality/>
 37. The great toil shift: How AI is redefining technical debt - Sonar, hämtad maj 12, 2026, <https://www.sonarsource.com/blog/how-ai-is-redefining-technical-debt>
 38. Measuring and Identifying Code-level Technical Debt: A Practical Guide - Sonar, hämtad maj 12, 2026,
<https://www.sonarsource.com/resources/library/measuring-and-identifying-code-level-technical-debt-a-practical-guide/>
 39. Semgrep App Security Platform | AI-assisted SAST, SCA and Secrets Detection, hämtad maj 12, 2026, <https://semgrep.dev/>
 40. Secure your code at scale with AI-driven vulnerability management - Datadog, hämtad maj 12, 2026,
<https://www.datadoghq.com/blog/code-security-ai-capabilities/>

41. Best AI Evaluation Tools for Prompt Experimentation in 2026 - Confident AI, hämtad maj 12, 2026, <https://www.confident-ai.com/knowledge-base/compare/best-ai-evaluation-tools-for-prompt-experimentation-2026>
42. AI Coding Statistics 2026 — Adoption, Productivity, Trust & Market Metrics - Panto AI, hämtad maj 12, 2026, <https://www.getpanto.ai/blog/ai-coding-assistant-statistics>
43. Manage costs effectively - Claude Code Docs, hämtad maj 12, 2026, <https://code.claude.com/docs/en/costs>
44. Quantifying Developer Productivity Gains from AI Tools - MetaCTO, hämtad maj 12, 2026, <https://www.metacto.com/blogs/quantifying-developer-productivity-gains-from-ai-tools>
45. AI Coding ROI Calculator 2025: Quantify Development Returns, hämtad maj 12, 2026, <https://www.augmentcode.com/tools/ai-coding-roi-calculator-2025-quantify-development-returns>
46. Developer ramp-up time continues to accelerate with AI - DX, hämtad maj 12, 2026, <https://getdx.com/blog/developer-ramp-up-time-continues-to-accelerate-with-ai/>
47. Developer Onboarding Best Practices: From Offer to Full Productivity - daily.dev, hämtad maj 12, 2026, <https://daily.dev/blog/developer-onboarding-best-practices-offer-to-full-productivity/>
48. How to Build Onboarding Time Tracking for Platform Engineering - OneUptime, hämtad maj 12, 2026, <https://oneuptime.com/blog/post/2026-01-30-platform-eng-onboarding-time/view>
49. Developer Onboarding: Faster Ramp Up, Higher Retention - Enboarder, hämtad maj 12, 2026, <https://enboarder.com/blog/developer-onboarding/>
50. 2026 Agentic Coding Trends Report - Anthropic, hämtad maj 12, 2026, <https://resources.anthropic.com/hubfs/2026%20Agentic%20Coding%20Trends%20Report.pdf>
51. When AI Code Errors Compound: A Math Check on Dependency Risk | zircote, hämtad maj 12, 2026, <https://zircote.com/blog/2026/05/ai-code-error-compounding-in-service-dependencies/>
52. Top AI Platforms Managing AI Token Level Usage Costs 1Afca - Prompts.ai, hämtad maj 12, 2026, <https://www.prompts.ai/blog/top-ai-platforms-managing-ai-token-level-usage-costs-1afca.html>
53. Top 10 AI Cost Visibility Tools in 2026 - Holori, hämtad maj 12, 2026, <https://holori.com/top-10-ai-cost-visibility-tools-in-2026/>
54. What Is Comprehension Debt in Software Development? - MadeCurious, hämtad

maj 12, 2026, <https://madecurious.com/articles/what-is-comprehension-debt/>
55. The State of Organizations 2026 | McKinsey, hämtad maj 12, 2026,
<https://www.mckinsey.com/~media/mckinsey/business%20functions/people%20and%20organizational%20performance/our%20insights/the%20state%20of%20organizations/2026/the-state-of-organizations-2026.pdf>